

Sri Lanka Travel Agent

Architecture & Conversation Guide

Generated: 2026-08-12 11:32

Dataset-backed travel planner (350 attractions + 33K reviews)

This document explains how the project is structured, how data flows from your Excel/CSV datasets into recommendations and itineraries, and how each user message is processed through the conversation engine - including category tabs and mood_tag-based planning from attractions.xlsx.

1. System Overview

The application is a multi-turn Sri Lanka trip planner. Users chat in a Streamlit UI. The FastAPI backend runs a deterministic AgentOrchestrator that collects trip preferences, searches the attractions database, and builds day-by-day itineraries. Places and descriptions always come from the loaded dataset; reviews provide supporting evidence from ChromaDB.

Technology stack

- Frontend: Streamlit (frontend/app.py) - ChatGPT-style chat UI

- API: FastAPI

Dataset sources

- attractions.xlsx - 350 rows: Attraction Name, Category, Destination, Details, mood_tag, Image

- Destination: Sri Lanka

2. Architecture Diagram

+-----

Data ingestion pipeline (offline)

attracti

3. End-to-End Request Flow

Every user message follows this path:

- 1. User types in Streamlit chat, clicks a welcome starter chip, or a sidebar category tab.

- 2. fronte

4. How Conversations Work

ConversationState (memory between turns)

Stored as JSON in the last assistant message metadata. Key fields:

- category_tag / category_confirmed - Wild, Heritage, etc. (hard filter like sidebar tab)

- mood_tag

Dual filters: Category and mood_tag

Both columns come from attractions.xlsx. When confirmed, they behave like hard SQL filters - only matching rows are suggested or scheduled. Category and mood can combine (e.g. Heritage tab + 'I feel curious' -> Heritage AND Curious).

category

Planning flows

Flow A - Category tab (sidebar Wild / Heritage / ...):

User cli

Flow B - Mood-first planning (user states mood):

User: "I

Flow C - Welcome starter chips (home screen):

User cli

Flow D - Open planning (user describes trip):

User: "P

Flow E - Replan by mood after itinerary:

[After i

Flow F - Bare category after greeting:

User: "H

Intent detection (agent_intent.py)

- GREETING - hi, hello (no active session)

- START_

Mood phrase parsing (planning_input.py + planning_flow.py)

- normalize_mood_feel_phrases() - fixes 'ifeelcurious' -> 'i feel curious'

- apply_m

5. How Recommendations Work

RecommendationService (recommendation_service.py) is attraction-first:

- Step 1: AttractionRepository.search() queries DB by category, mood_tag, district

- Step 2: f

Itinerary building

ItineraryPlanner (itinerary_planner.py) takes ranked attractions and schedules them across days using district centroids (geography.py) for travel compatibility. ItineraryService formats the plan into readable day headers and place cards.

6. File-by-File Guide

API & Frontend

`api/main.py`

FastAPI app

`frontend/app.py`

Streamlit UI

Conversation core

`src/services/chat_service.py`

Thin facade

`src/services/agent_orchestrator.py`

Main turn ro

`src/services/agent_intent.py`

Detect user

`src/services/agent_handlers.py`

Decline, reje

`src/services/agent_tools.py`

search_attra

`src/services/conversation_state.py`

Dataclass ho

`src/services/conversation_service.py`

Persist mess

`src/services/state_manager.py`

Parse signal

Planning

`src/services/planning_flow.py`

Planning ste

`src/services/planning_input.py`

Parse durati

`src/services/preferences.py`

VALID_CAT

`src/services/geography.py`

District coor

Recommendations & itineraries

`src/services/recommendation_service.py`

Rank attract

`src/services/itinerary_planner.py`

Geo-aware s

`src/services/itinerary_service.py`

Format itine

`src/services/attraction_cards.py`

Markdown/H

`src/services/category_filter.py`

Hard categor

Reviews & evidence

`src/services/review_evidence.py`

Fetch Chrom

`src/services/review_stats.py`

Cached revi

`src/vectorstore/chroma_store.py`

ChromaDB p

Database

[src/database/models.py](#)

[src/database/postgres.py](#)

[src/database/repositories.py](#)

Data pipeline

[src/data/loader.py](#)

[src/data/cleaner.py](#)

[src/data/matcher.py](#)

[src/data/normalizer.py](#)

[scripts/clean_data.py](#)

[scripts/load_postgres.py](#)

[scripts/build_vector_db.py](#)

Agent (secondary path)

[src/agent/graph.py](#)

[src/agent/prompts.py](#)

[src/tools/__init__.py](#)

[src/services/conversation_router.py](#)

Auth & config

[src/auth/service.py](#)

[src/config.py](#)

Sentence-tra

SQLAlchem

Engine and

AttractionRe

Read raw C

Normalize a

Link attractio

District/nam

Phase 1: raw

Phase 2: pro

Phase 3: em

LangGraph

System prom

LangChain t

LLM router f

Register, log

Paths, DB U

7. Example Conversation Traces

Example 1: Wild trip from sidebar

Turn 1 US

Example 2: Mood-first planning (Curious)

Turn 1 US

Example 3: Welcome chip + near district

Turn 1 US

Example 4: Replan itinerary by mood

Turn N BO

Example 5: Open planning with wildlife interest

Turn 1 US

Example 6: Build itinerary from recommendations

Turn N BO

8. Running the System

Initial setup

```
pip instal
```

Health check

GET <http://localhost:8000/health> - shows chroma_review_count and LLM config

GET <http://localhost:8000/dataset/categories> - category counts from DB

Key design rules

- Attractions are NEVER invented - only the 350 loaded rows can appear

- Reviews